```
function createMap(countries, cities) {
  var width = 500;
  var height = 500;
  var projection = d3.geo.mercator()
                     .scale(80)
                     .translate([width / 2, height / 2]);
  var geoPath = d3.geo.path().projection(projection);

  d3.select("svg").selectAll("path").data(countries.features)
     .enter()
     .append("path")
     .attr("d", geoPath)
     .style("fill", "gray");

  d3.select("svg").selectAll("circle").data(cities)
     .enter()
     .append("circle")
     .style("fill", "red")
     .attr("class", "cities")
     .attr("r", 3)
     .attr("cx", function(d) {return projection([d.x,d.y])[0]})
     .attr("cy", function(d) {return projection([d.x,d.y])[1]});
};
```

Overrides the fill style so it'll be easier to see your cities

You want to draw the cities over the countries, so you append them second.

Projection returns an array, which means you need to take the [0] value for cx and the [1] value for cy

One thing to note from listing 7.6 is that coordinates are often given in the real world in the order of "latitude, longitude." Because latitude corresponds to the y-axis and longitude corresponds to the x-axis, you have to flip them to provide the x, y coordinates necessary for GeoJSON and D3.

7.1.3 Projections and areas

Depending on what projection you use, the graphical size of your geographic objects will appear different. This is because it's impossible to perfectly display spherical coordinates on a flat surface. Different projections are designed to visually display the geographic area of land or ocean regions, or the measurable distance, or particular shapes. Because we included `d3.geo.projection.js`, we have access to quite a few more projections to play with, one of which is the Mollweide projection. In the code in listing 7.7, you can see the settings necessary to properly display a Mollweide projection of our geodata. We'll use the calculated area of the countries (the graphical area, not their actual physical area) to color each country. The results are quite distinct from the same code running on our Mercator projection, as shown in figure 7.6. The world as displayed with Mollweide curves the edges, rather than stretching them into a rectangle like Mercator does.

Listing 7.7 Mollweide projected world

```
queue()
  .defer(d3.json, "world.geojson")
  .defer(d3.csv, "cities.csv")
  .await(function(error, file1, file2) {
           createMap(file1, file2);
   });
```